44

Sosyal Medya Haber Akışı (News Feed / Twitter): Fan-Out-on-Write vs Fan-Out-on-Read

Designing a News Feed (Twitter / Instagram): Fan-Out Architectures

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. FAN-OUT-ON-WRITE (Push Modeli - Normal Kullanıcılar İçin) ]
Ali Tweet Atar ──► Takipçileri Bul (150 kişi) ──► Her birinin Feed Önbelleğine Yaz!
(Okuma anında akış hazır bekler: O(1) okuma hızı!)

[ 2. FAN-OUT-ON-READ (Pull Modeli - Ünlüler / Celebrities İçin) ]
Cristiano Ronaldo Tweet Atar (600 Milyon Takipçi!)
600 Milyon önbelleğe yazmaya kalkarsanız sunucular çöker (Celebrity Problem)!
Çözüm: Ronaldo'nun tweet'i sadece kendi profiline yazılır. Takipçi akışını açtığında
normal akış ile Ronaldo'nun tweet'i dinamik olarak harmanlanır (Merge)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Haber akışı oluşturmanın iki yolu vardır: 1) Fan-Out-on-Write (Gazete Dağıtıcısı Modeli): Mahallenin muhtarı bir duyuru yaptığında, çırak gidip mahalledeki 50 evin kapısının altından duyuruyu atar. Sabah uyanan herkes kapının önündeki duyuruyu saniyesinde okur (Okuma anında sıfır bekleme!). 2) Fan-Out-on-Read (Ünlü Problemi): Eğer Cristiano Ronaldo veya Taylor Swift duyuru yaparsa ne olur? 600 milyon takipçinin evine tek tek kağıt dağıtmaya kalkarsanız posta teşkilatı iflas eder! Ünlüler duyuruyu meydana asar; siz sabah sokağa çıktığınızda kendi mektuplarınızla meydandaki büyük afişi birleştirip okursunuz!

🇬🇧 ENGLISH (ELI5)

Two contrasting architectures for social feeds: 1) Fan-Out-on-Write (Push Model): When an ordinary user with 200 friends posts a photo, background workers push a reference into each of those 200 friend's private in-memory feed caches. When a friend opens their phone, their timeline is already pre-computed (O(1) read latency!). 2) Fan-Out-on-Read (Pull Model / The Celebrity Problem): When Cristiano Ronaldo or Taylor Swift posts to 600 million followers, pushing to 600M individual caches burns terabytes of RAM and locks queues for hours! Instead, store the celebrity tweet once on their wall; when a fan opens their feed, the app dynamically merges their friends' cached feed with Ronaldo's latest tweet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG News Feed Mimarisi (Hibrit Model): 1) Fan-Out-on-Write (Push): Normal kullanıcılar (<25.000 takipçi) için geçerlidir. Kullanıcı tweet attığında bir worker takipçilerin Redis ZSET akışlarına tweet ID'sini ekler. Okuma $O(1)$'dir. 2) Fan-Out-on-Read (Pull): Ünlüler (Celebrity / VIP kullanıcılar) için geçerlidir. Tweet takipçilerin önbelleğine yazılmaz; sadece ünlünün kendi kütüğüne yazılır. İstemci akışını yenilediğinde, önbelleğindeki akış ile takip ettiği ünlülerin en son tweetleri birleştirilerek (K-Way Merge) kullanıcıya sunulur. Bu hibrit mimari hem Twitter'ın hem de Instagram'ın omurgasıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Hybrid News Feed Architecture: 1) Fan-Out-on-Write (Push): Applied to standard users (<25,000 followers). Asynchronous workers fan out the post ID into follower timeline caches (Redis ZSET keyed by timestamp). Delivers $O(1)$ instantaneous feed generation. 2) Fan-Out-on-Read (Pull): Applied to high-degree celebrities (e.g. 50M+ followers). Writing to 50M Redis sets exhausts cluster bandwidth. The celebrity tweet is stored only in their user timeline; when an active follower requests their feed, the system executes an in-memory K-way merge between the user's pre-computed inbox and their followed celebrities' latest posts. This hybrid approach represents the production standard at Twitter and Instagram.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Hibrit Fan-Out Karar Motoru (Celebrity Problem Çözümü)
public class HybridFeedFanOut {
    private static final int CELEBRITY_THRESHOLD = 50_000;

    public record User(String id, String username, int followerCount) {
        public boolean isCelebrity() { return followerCount >= CELEBRITY_THRESHOLD; }
    }

    public String determineFanOutStrategy(User author) {
        if (author.isCelebrity()) {
            return "PULL_STRATEGY: Takipçilere yazma! Sadece ünlünün profiline kaydet. (Followers: " +
author.followerCount() + ")";
        } else {
            return "PUSH_STRATEGY: " + author.followerCount() + " takipçinin Redis Feed önbelleğine
asenkron fırlat!";
        }
    }
}

void main() {
    var engine = new HybridFeedFanOut();
    var normalUser = new HybridFeedFanOut.User("u1", "ali_bozkurt", 240);
    var celebrity = new HybridFeedFanOut.User("u2", "elon_musk", 180_000_000);

    IO.println("Ali için Strateji:  " + engine.determineFanOutStrategy(normalUser));
    IO.println("Elon için Strateji: " + engine.determineFanOutStrategy(celebrity));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Her tweet atıldığında takipçilerin feed'ini yeniden hesaplarım (Fan-out on Read)' derseniz sistem 100M kullanıcıda çöker; 'Herkes için Fan-out on Write yaparım' derseniz Elon Musk tweet attığında sistem çöker! Çözüm: Kesinlikle HİBRİT modeldir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Choosing purely Fan-out on Read causes database exhaustion during timeline loads; choosing purely Fan-out on Write destroys queues when celebrities post. Propose the HYBRID model: push for ordinary users, pull for celebrities.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Fan-out on Write okuma hızını O(1) seviyesine indirir ancak ünlülerde patlar.
- Celebrity Problem, takipçi sayısı yüksek hesapların akışını okuma anında dinamik harmanlayarak çözülür.
- Haber akışları Redis Sorted Sets (ZSET) veri yapısında zaman damgası skoruyla tutulur.

🎯 Key Takeaways for the Interview

- Fan-out on Write provides instantaneous O(1) timeline reads but stumbles on celebrity fan-out.
- The Celebrity Problem is resolved by dynamically blending hot celebrity timelines on read.
- User feed inboxes are maintained inside Redis Sorted Sets keyed by epoch millisecond scores.